



UNINASSAU

INTRODUÇÃO AO JAVA E AO ECLIPSE IDE

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

INTRODUÇÃO AO JAVA

O QUE A LINGUAGEM DE PROGRAMAÇÃO JAVA?

A linguagem de programação Java é uma linguagem de alto nível, orientada a objetos e de propósito geral, criada pela Sun Microsystems em 1995 (hoje mantida pela Oracle). Ela é amplamente utilizada no desenvolvimento de aplicações para computadores, web, dispositivos móveis (especialmente Android), sistemas embarcados e muito mais.

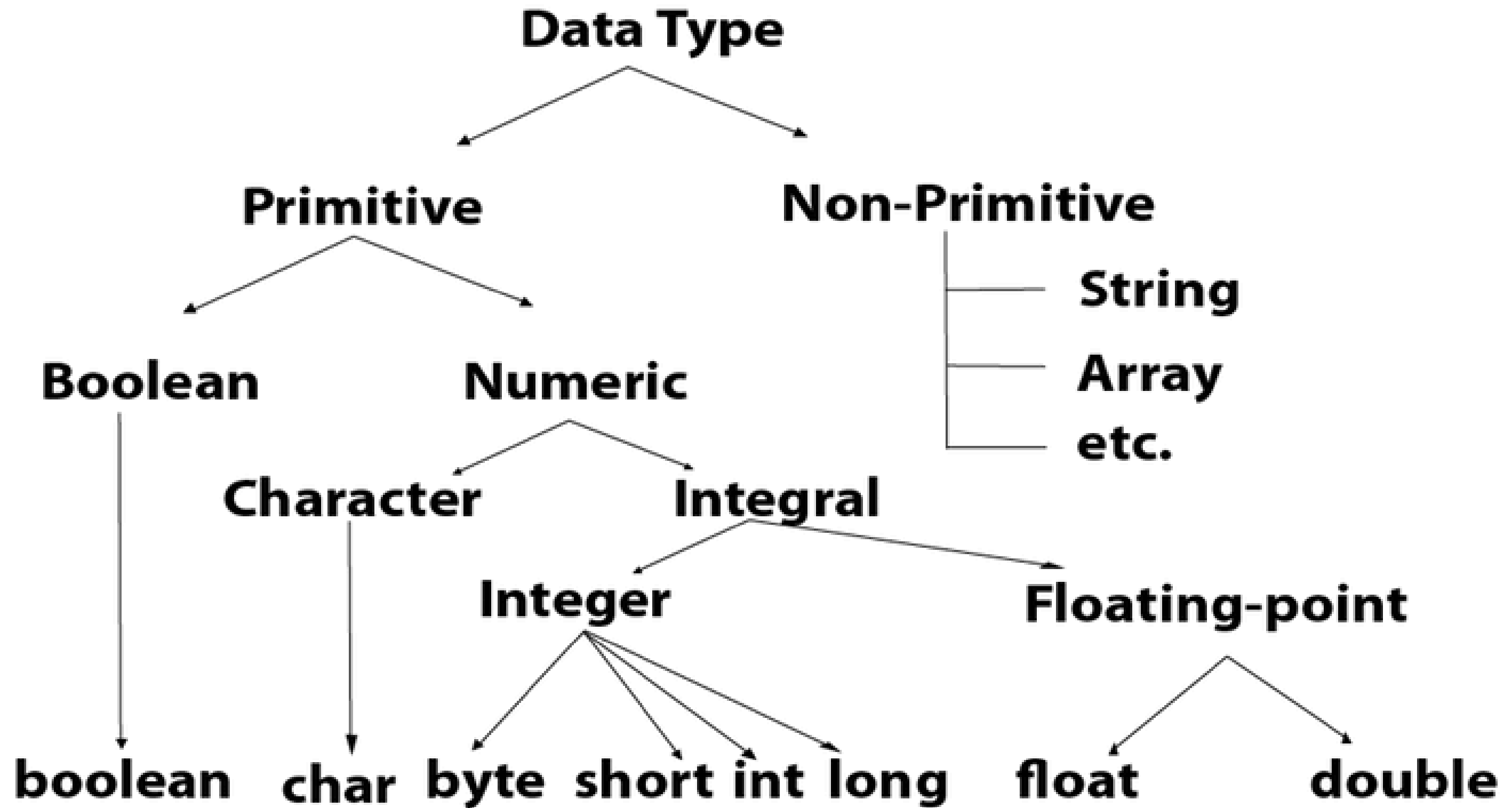


ONDE O JAVA É UTILIZADO?

- Aplicações desktop (Swing, JavaFX)
- Desenvolvimento Android (com Android Studio)
- Aplicações web (com Java EE, Spring)
- Sistemas corporativos e bancos
- Dispositivos embarcados (IoT)



Tipos dados em Java



Tipos de Dados Primitivos

Os tipos de dados primitivos em Java representam os valores mais básicos e fundamentais que a linguagem pode manipular.

Tipo	Exemplo	Descrição
<code>int</code>	10	Números inteiros
<code>double</code>	3.14	Números decimais (ponto flutuante)
<code>char</code>	'A'	Um único caractere
<code>boolean</code>	true / false	Verdadeiro ou falso
<code>float</code>	3.14f	Versão menor de <code>double</code>
<code>long</code>	123456789L	Inteiros muito grandes
<code>byte</code>	127	Números pequenos (8 bits)
<code>short</code>	32000	Números inteiros curtos

Tipos de referência (objetos):

Tipo	Exemplo	Descrição
<code>String</code>	<code>"Olá"</code>	Cadeia de caracteres
<code>Scanner</code>	<code>new Scanner(System.in)</code>	Objeto para entrada de dados
<code>Array</code> , <code>List</code> , <code>Object</code> , etc.	Usado para armazenar múltiplos valores ou objetos	

SINTAXE DO JAVA

1. Declaração de variáveis

```
1  int idade = 25;           // inteiro
2  double altura = 1.75;     // número decimal
3  boolean ativo = true;     // valor lógico
4  char letra = 'A';         // caractere único
5  String nome = "Maria";    // texto (cadeia de caracteres)
6
```

2. Estrutura condicional (if / else)

```
1  int numero = 10;
2
3  ✓ if (numero % 2 == 0) {
4      System.out.println("Par");
5  ✓ } else {
6      System.out.println("Ímpar");
7  }
8
```

3. Laço de repetição (for)

```
1  ✓ for (int i = 0; i < 5; i++) {  
2      System.out.println("Contando: " + i);  
3  }  
4
```

4. Laço de repetição (while)

```
1  int contador = 0;
2
3  ✓ while (contador < 5) {
4      System.out.println("Repetição " + contador);
5      contador++;
6  }
7
```

5. Entrada de dados com Scanner

```
1  import java.util.Scanner;
2
3  ✓ public class Entrada {
4  ✓      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          System.out.print("Digite seu nome: ");
8          String nome = sc.nextLine();
9
10         System.out.println("Olá, " + nome + "!");
11
12         sc.close();
13     }
14 }
15
```


6. Declaração de método (função)

```
1  ✓ public static int somar(int a, int b) {  
2      return a + b;  
3  }  
4
```

7. Criação de classe e objeto

```
1  ✓ public class Pessoa {  
2      String nome;  
3      int idade;  
4  
5  ✓  void apresentar() {  
6      System.out.println("Olá, meu nome é " + nome + " e tenho " + idade + " anos.");  
7  }  
8  }  
9
```

8. Uso de arrays (vetores e matrizes)

```
1  int[] numeros = {1, 2, 3, 4, 5};  
2  System.out.println(numeros[0]); // imprime 1  
3
```

```
1  int[][] matriz = new int[3][2]; // 3 linhas, 2 colunas  
2
```


8. Uso de arrays (vetores e matrizes)

Atribuição dos valores de uma matriz:

```
1  matriz[0][0] = 10;  
2  matriz[0][1] = 20;  
3  matriz[1][0] = 30;  
4  matriz[1][1] = 40;  
5  matriz[2][0] = 50;  
6  matriz[2][1] = 60;  
7
```

ECLIPSE IDE

O que é o IDE Eclipse?

O Eclipse é um Ambiente de Desenvolvimento Integrado (IDE) livre e de código aberto que é amplamente utilizado para desenvolver aplicações Java. É uma ferramenta poderosa que fornece uma variedade de recursos para escrever, depurar, testar e gerenciar projetos Java.



Java Development Kit (JDK)

O Java Development Kit (JDK) é um pacote de software fornecido pela Oracle (ou outros distribuidores, como o OpenJDK) que contém tudo o que você precisa para desenvolver programas em Java.

Instalar o JDK

- Baixar o JDK mais recente (Java Development Kit):
- <https://www.oracle.com/java/technologies/javase-downloads.html>
- ou usar o OpenJDK: <https://adoptium.net/>



Baixar o JDK mais recente (Java Development Kit)

ORACLE

Products Industries Resources Customers Partners Developers Company

🔍



 View Accounts

Java downloads

Tools and resources

Java archive

JDK 24

JDK 21

GraalVM for JDK 24

GraalVM for JDK 21

Java SE Development Kit 24.0.1 downloads

JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 24 will receive updates under these terms, until September 2025, when it will be superseded by JDK 25.

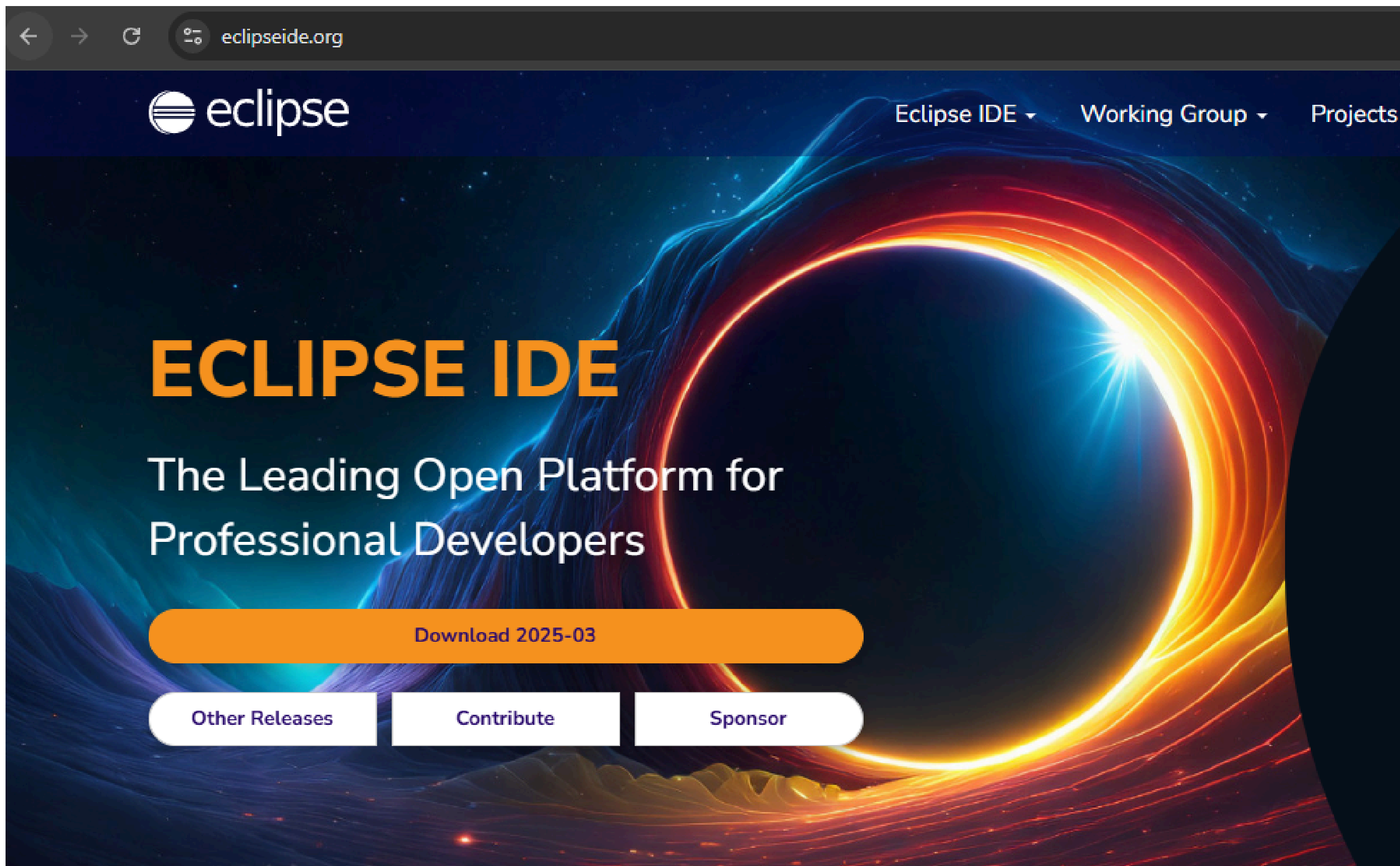
Linux

macOS

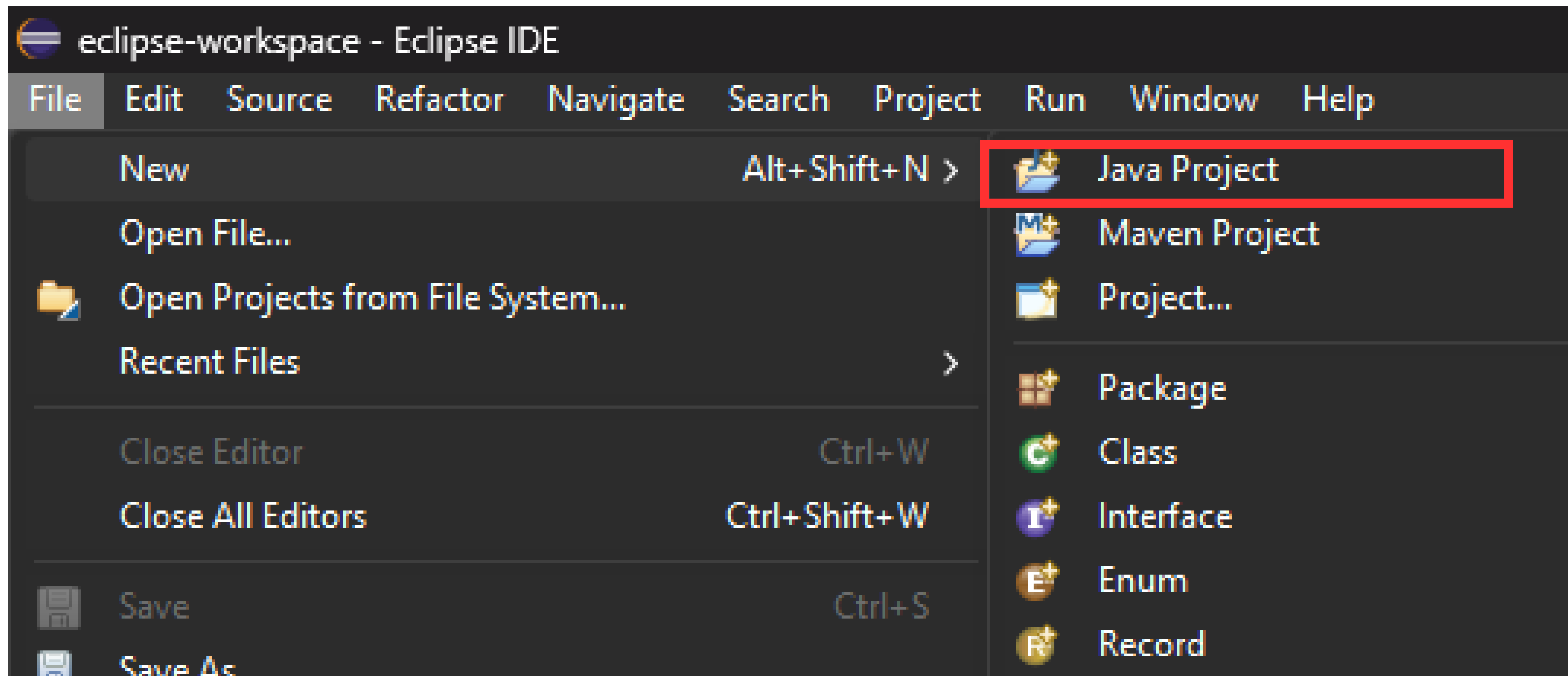
Windows

Product/file description	File size	Download
x64 Compressed Archive	229.51 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.zip (sha256)
x64 Installer	205.85 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.exe (sha256)
x64 MSI Installer	204.60 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.msi (sha256)

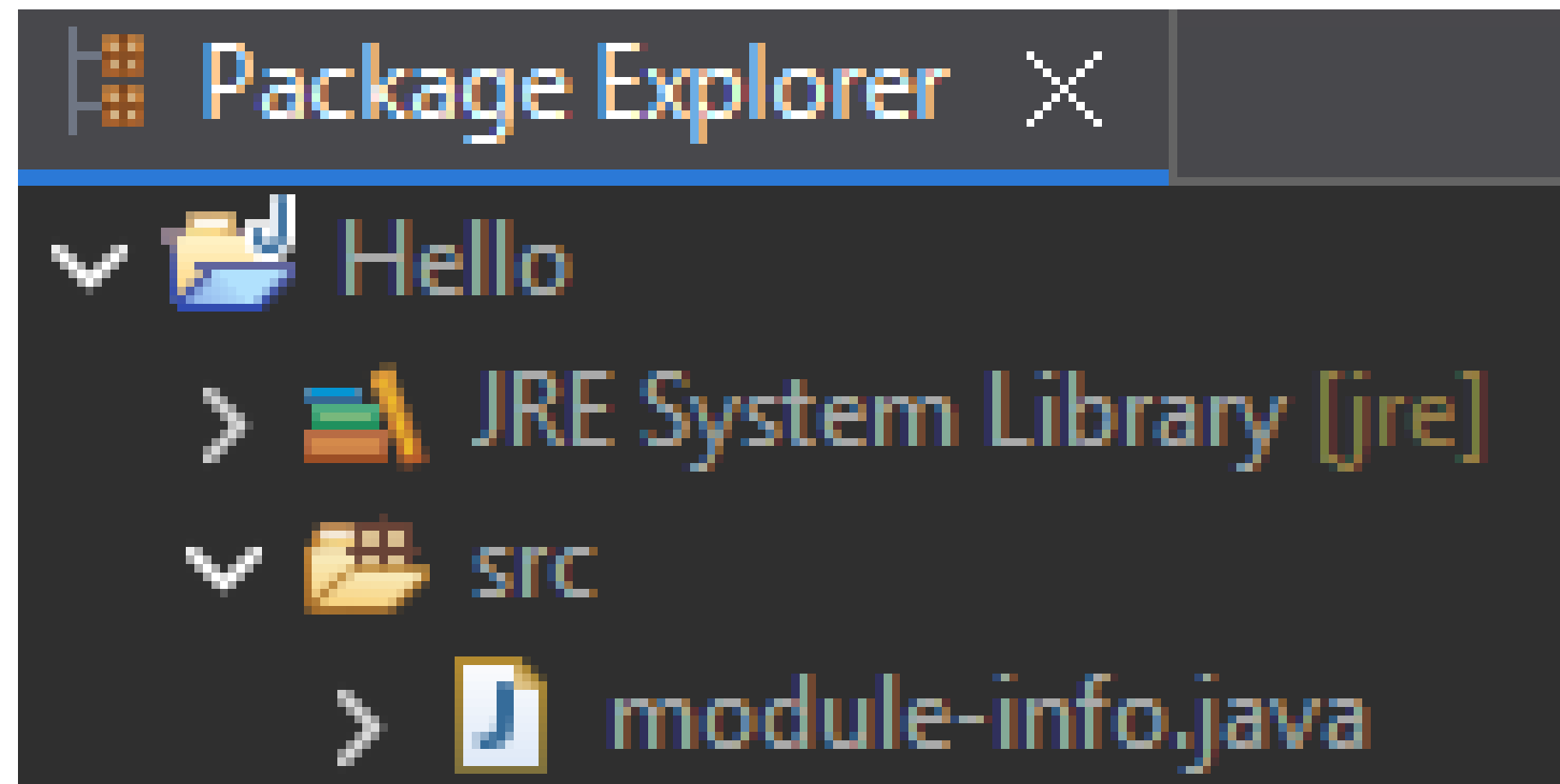
Baixando o Eclipse IDE em <https://eclipseide.org/>



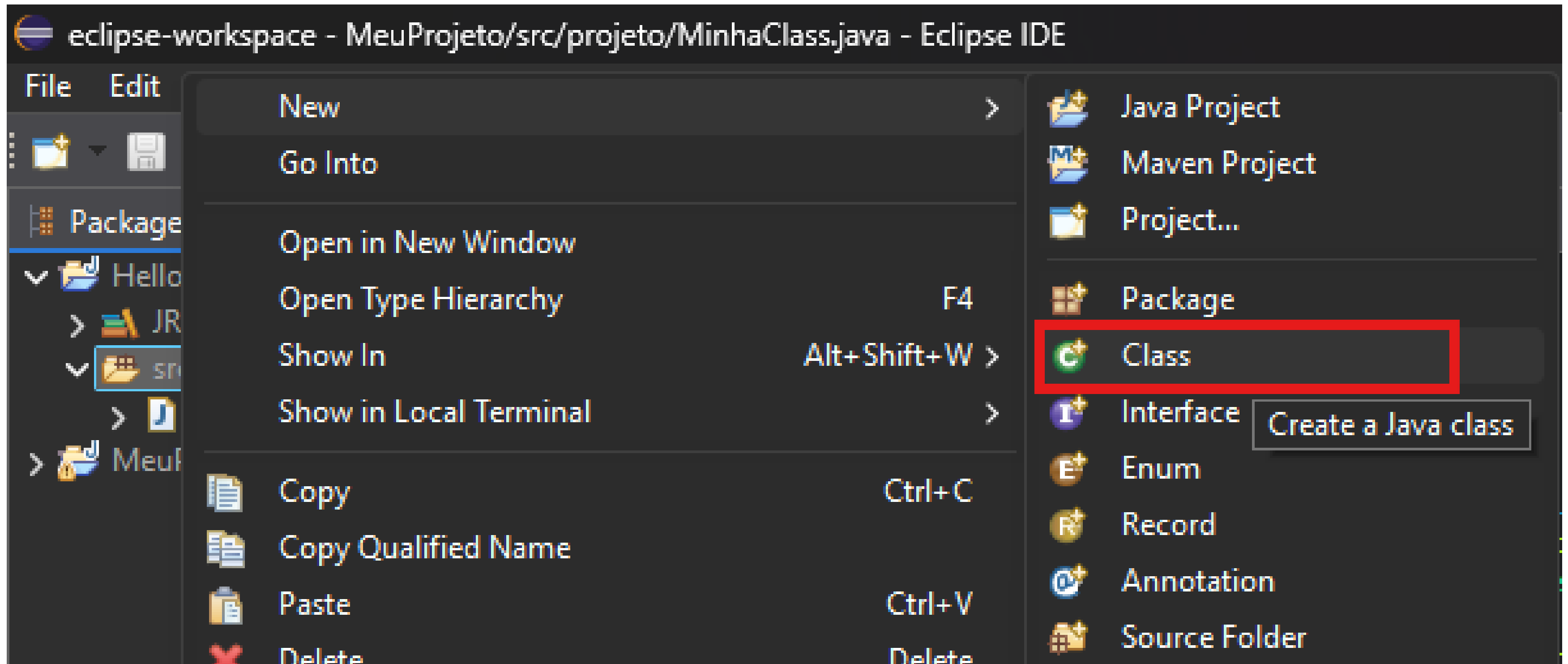
Criando o primeiro Projeto no Eclipse IDE



Criando o primeiro Projeto no Eclipse IDE



Criando o primeiro Projeto no Eclipse IDE



Criando o primeiro Projeto no Eclipse IDE

New Java Class

Java Class
Create a new Java class.

Source folder: Hello/src Browse...

Package: hello Browse...

☐ Enclosing type: Browse...

Name: Hello

Modifiers:
☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: java.lang.Object Browse...

Interfaces: Add...
Remove

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

? Finish Cancel

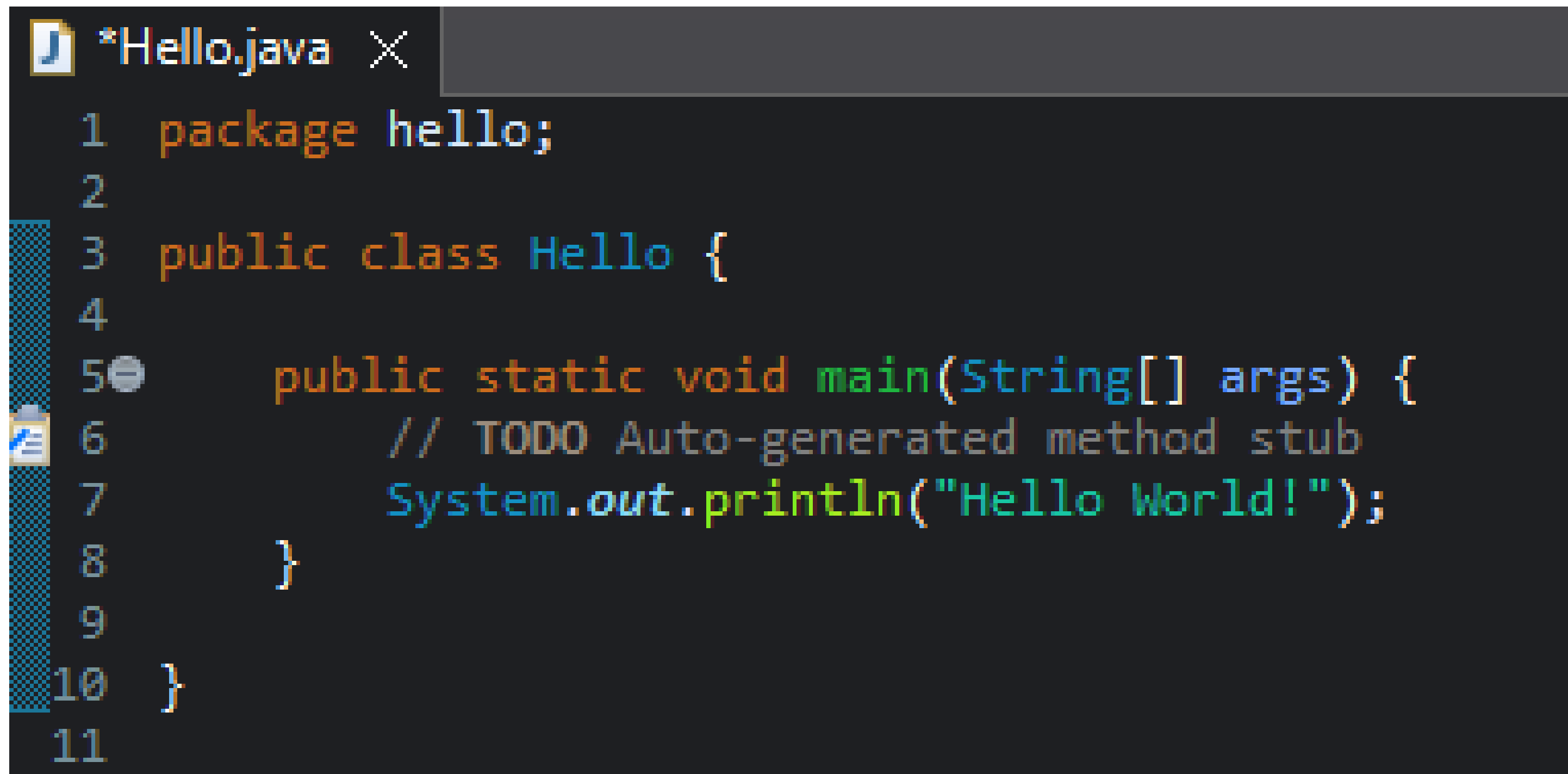
Which method stubs would you like to create?

☒ public static void main(String[] args)

☐ Constructors from superclass

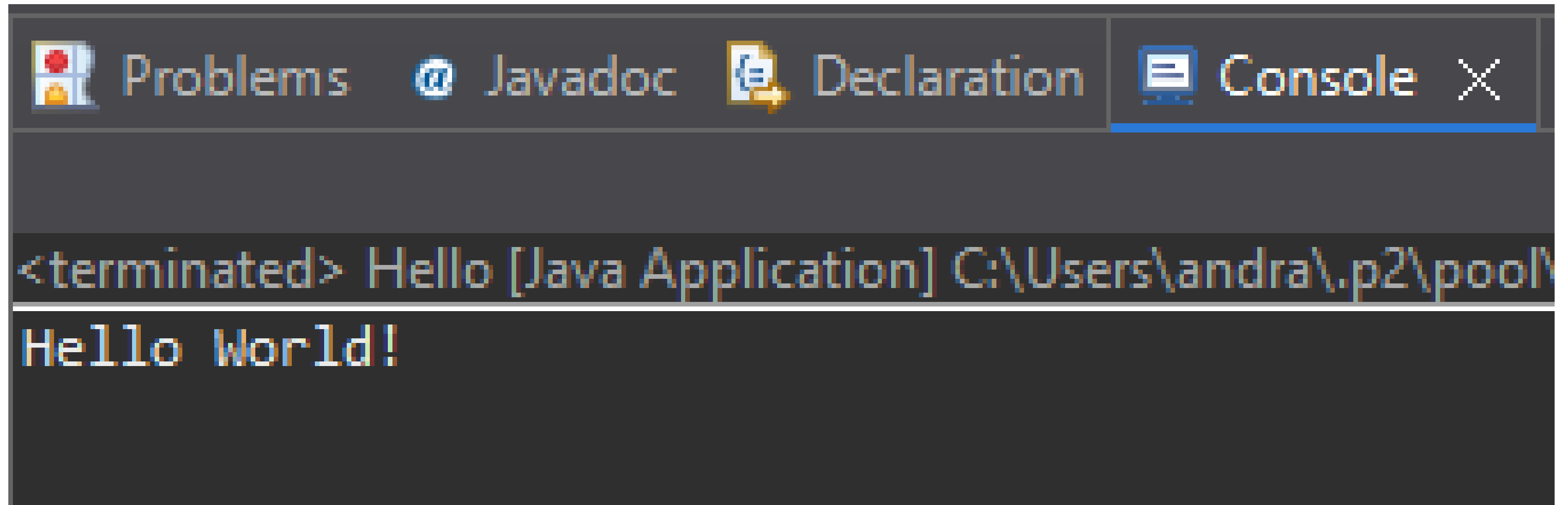
☒ Inherited abstract methods

Criando o primeiro Projeto no Eclipse IDE



```
*Hello.java X
1  package hello;
2
3  public class Hello {
4
5      public static void main(String[] args) {
6          // TODO Auto-generated method stub
7          System.out.println("Hello World!");
8      }
9
10 }
11
```

Criando o primeiro Projeto no Eclipse IDE



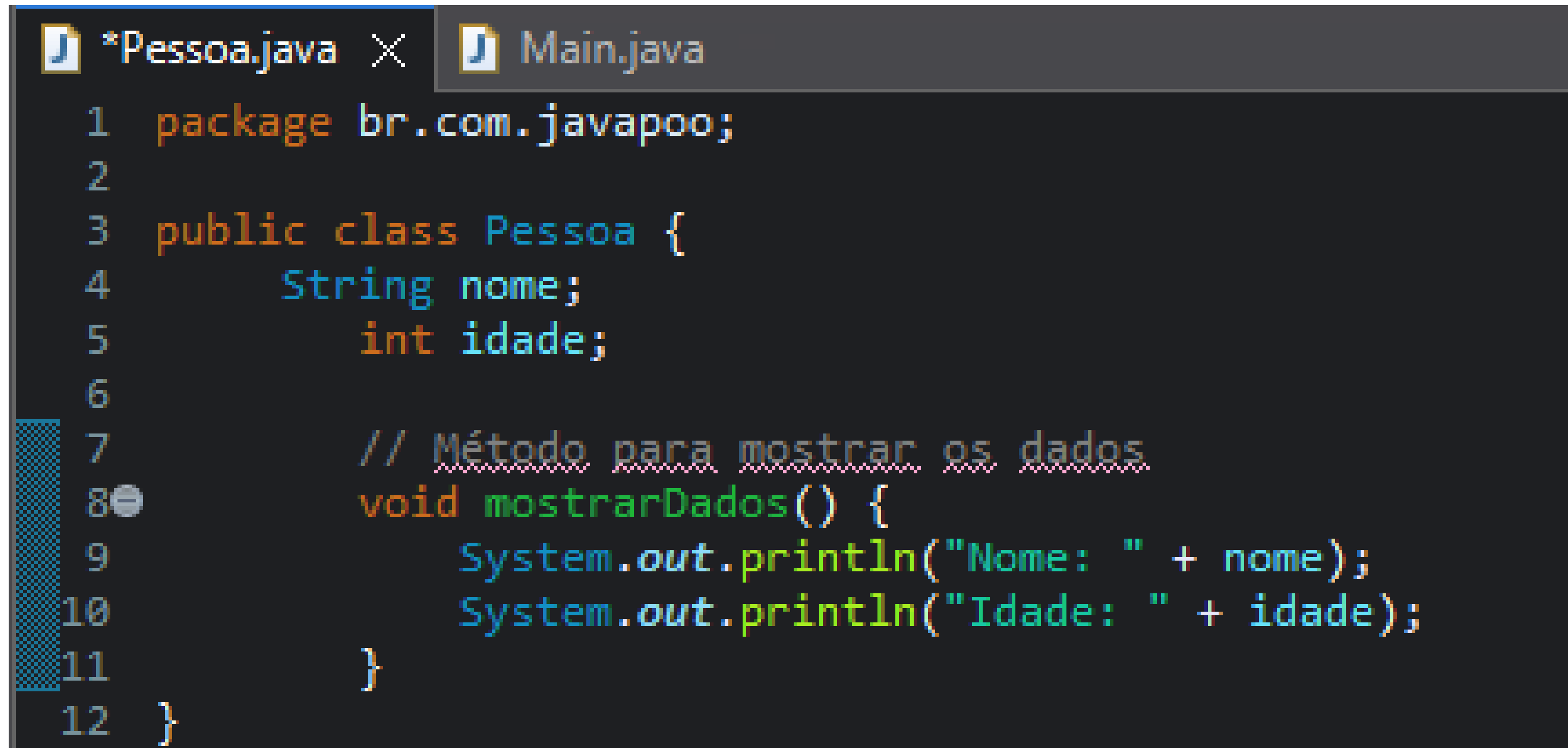
The screenshot shows the Eclipse IDE's Console window. The title bar at the top includes icons for Problems, Javadoc, Declaration, and Console, with the Console tab selected. The console output displays the text: `<terminated> Hello [Java Application] C:\Users\andra\.p2\pool\` followed by `Hello World!` on a new line.

```
<terminated> Hello [Java Application] C:\Users\andra\.p2\pool\  
Hello World!
```

POO EM JAVA

Exemplo 1: Classe e Objeto Simples

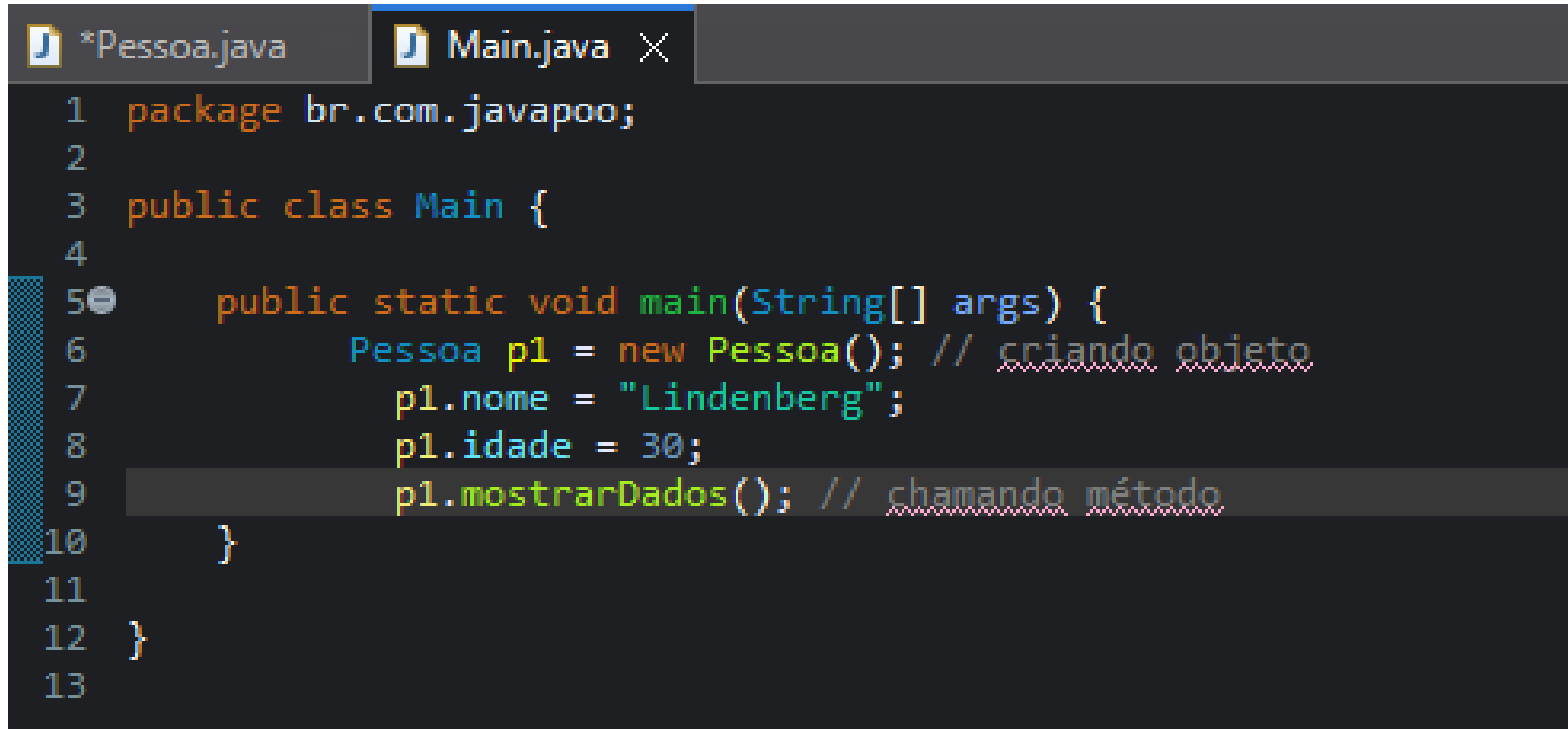
Objetivo: Criar uma classe Pessoa com atributos e métodos.



The screenshot shows a code editor with two tabs: `*Pessoa.java` and `Main.java`. The `Pessoa.java` tab is active, displaying the following Java code:

```
1 package br.com.javapoo;
2
3 public class Pessoa {
4     String nome;
5     int idade;
6
7     // Método para mostrar os dados
8     void mostrarDados() {
9         System.out.println("Nome: " + nome);
10        System.out.println("Idade: " + idade);
11    }
12 }
```

Exemplo 1: Classe e Objeto Simples



```
1 package br.com.javapoo;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         Pessoa p1 = new Pessoa(); // criando objeto
7         p1.nome = "Lindenberg";
8         p1.idade = 30;
9         p1.mostrarDados(); // chamando método
10    }
11
12 }
13
```

Exemplo 1: Classe e Objeto Simples

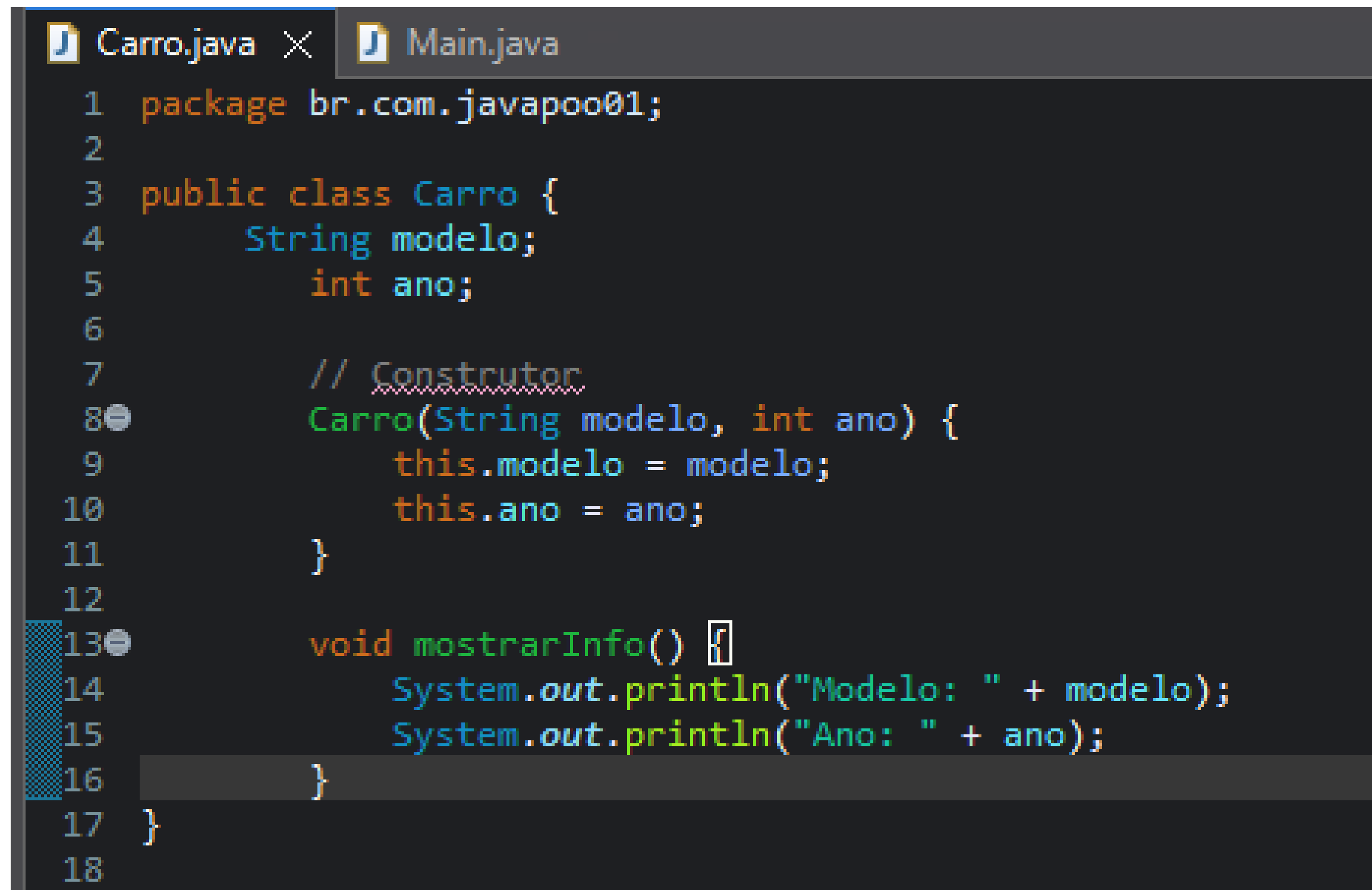
```
*Pessoa.java  Main.java X
1 package br.com.javapoo;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         Pessoa p1 = new Pessoa(); // criando objeto
7         p1.nome = "Lindenberg";
8         p1.idade = 30;
9         p1.mostrarDados(); // chamando método
10    }
11
12 }
13
```

Saída Esperada:

```
Nome: Lindenberg
Idade: 30
```

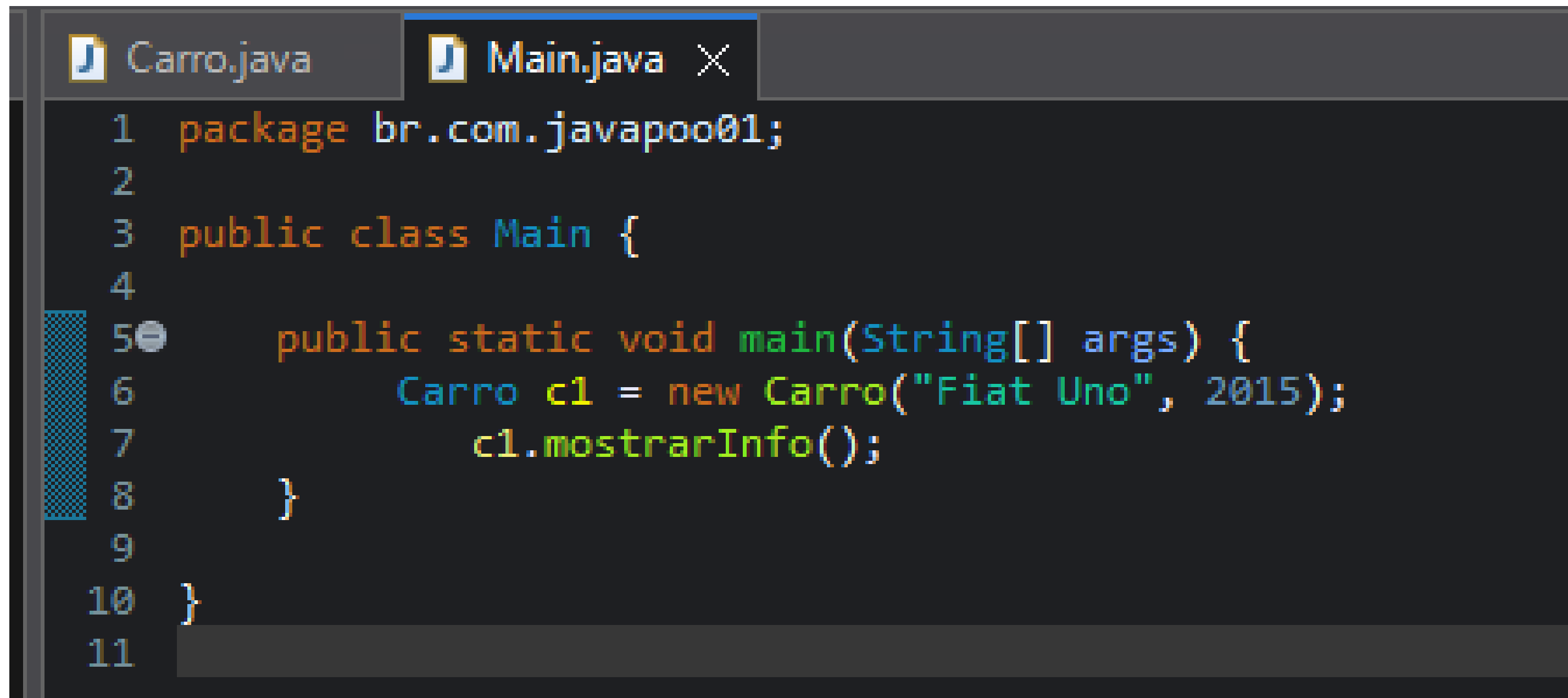

Exemplo 2: Classe com Construtor

Objetivo: Criar uma classe **Carro** que inicializa atributos pelo construtor.



```
Carro.java × Main.java
1 package br.com.javapoo01;
2
3 public class Carro {
4     String modelo;
5     int ano;
6
7     // Construtor
8     Carro(String modelo, int ano) {
9         this.modelo = modelo;
10        this.ano = ano;
11    }
12
13    void mostrarInfo() {
14        System.out.println("Modelo: " + modelo);
15        System.out.println("Ano: " + ano);
16    }
17 }
18
```

Exemplo 2: Classe com Construtor



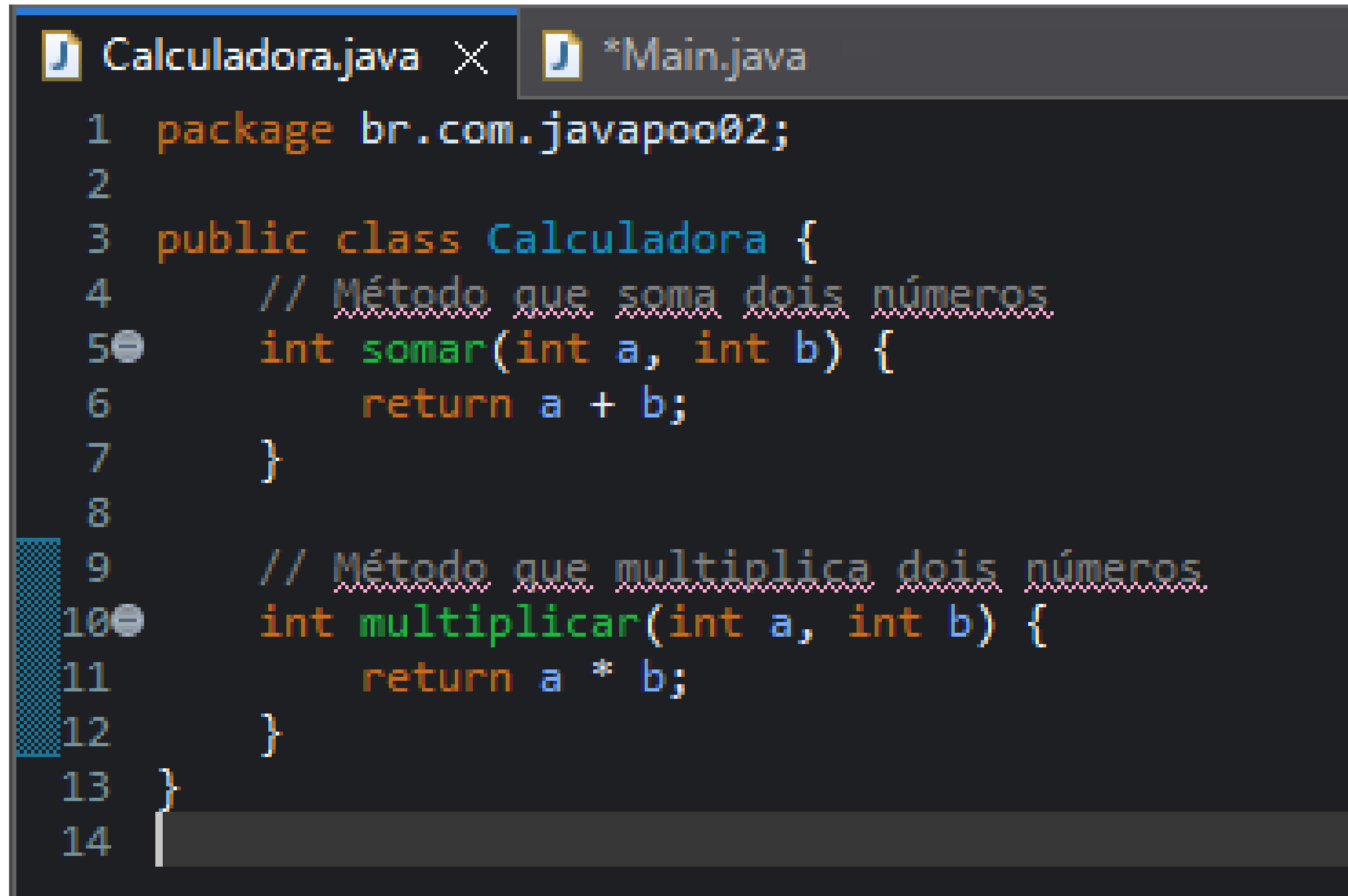
```
1 package br.com.javapoo01;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         Carro c1 = new Carro("Fiat Uno", 2015);
7         c1.mostrarInfo();
8     }
9
10 }
11
```

Saída Esperada:

```
Modelo: Fiat Uno
Ano: 2015
```

Exemplo 3: Métodos com Parâmetros e Retorno

Objetivo: Criar uma classe **Calculadora** que realiza operações matemáticas.



```
Calculadora.java X *Main.java
1 package br.com.javapoo02;
2
3 public class Calculadora {
4     // Método que soma dois números
5     int somar(int a, int b) {
6         return a + b;
7     }
8
9     // Método que multiplica dois números
10    int multiplicar(int a, int b) {
11        return a * b;
12    }
13 }
14
```

Exemplo 3: Métodos com Parâmetros e Retorno

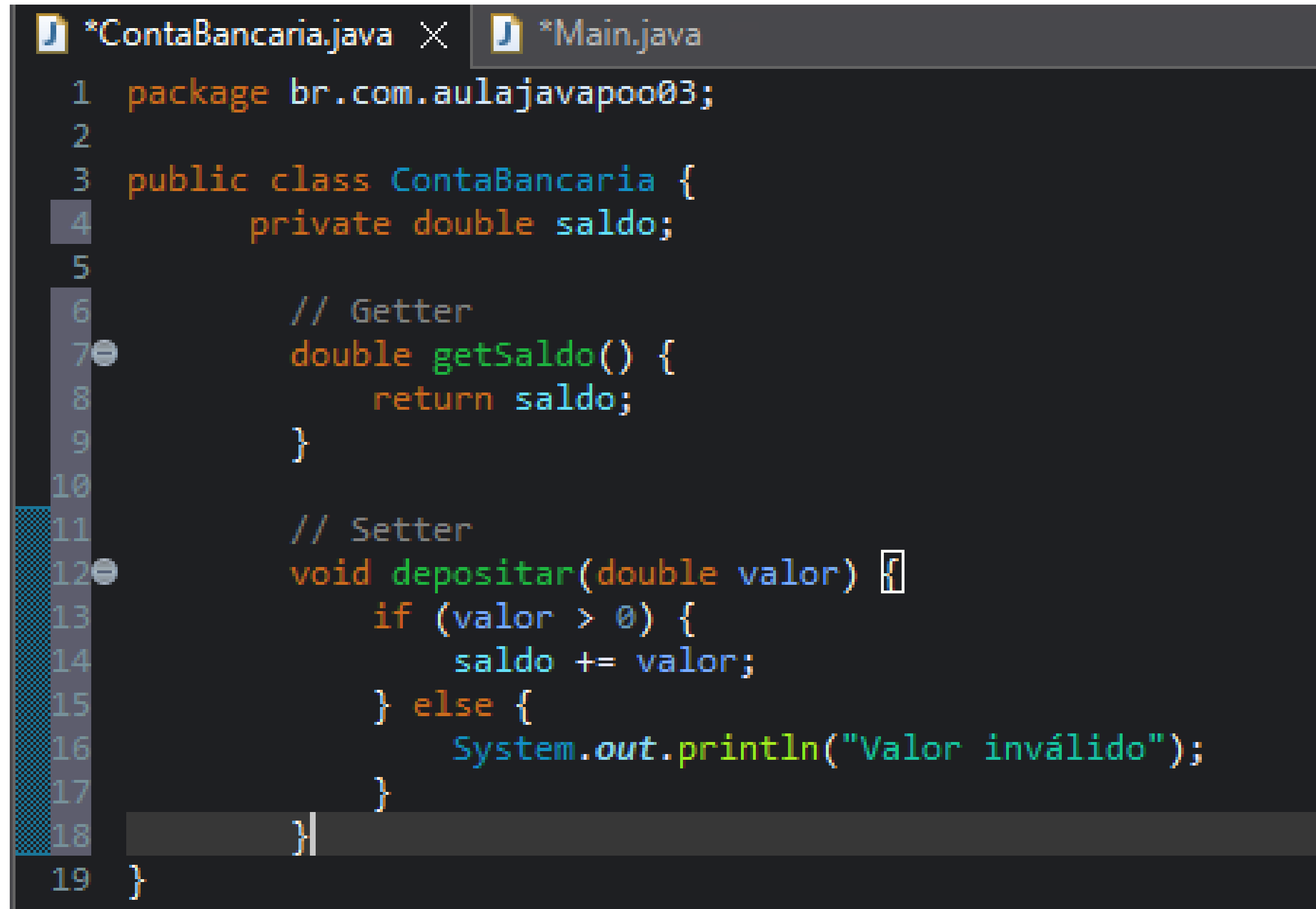
```
Calculadora.java  *Main.java X
1 package br.com.javapoo02;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         Calculadora calc = new Calculadora();
7         int resultadoSoma = calc.somar(5, 3);
8         int resultadoMultiplicacao = calc.multiplicar(4, 2);
9
10        System.out.println("Soma: " + resultadoSoma);
11        System.out.println("Multiplicação: " + resultadoMultiplicacao);
12    }
13
14
15 }
16
```

Saída Esperada:

```
Soma: 8
Multiplicação: 8
```

Exemplo 4: Encapsulamento com Getters e Setters

Objetivo: Proteger atributos usando **private** e acessar via métodos.



```
*ContaBancaria.java × *Main.java
1 package br.com.aulajavapoo03;
2
3 public class ContaBancaria {
4     private double saldo;
5
6     // Getter
7     double getSaldo() {
8         return saldo;
9     }
10
11    // Setter
12    void depositar(double valor) {
13        if (valor > 0) {
14            saldo += valor;
15        } else {
16            System.out.println("Valor inválido");
17        }
18    }
19 }
```


Exemplo 4: Encapsulamento com Getters e Setters

```
*ContaBancaria.java  *Main.java X
1 package br.com.aulajavapoo03;
2
3 public class Main {
4     public static void main(String[] args) {
5         ContaBancaria conta = new ContaBancaria();
6         conta.depositar(500);
7         System.out.println("Saldo: " + conta.getSaldo());
8     }
9 }
10
```

Saída Esperada: `Saldo: 500.0`

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code